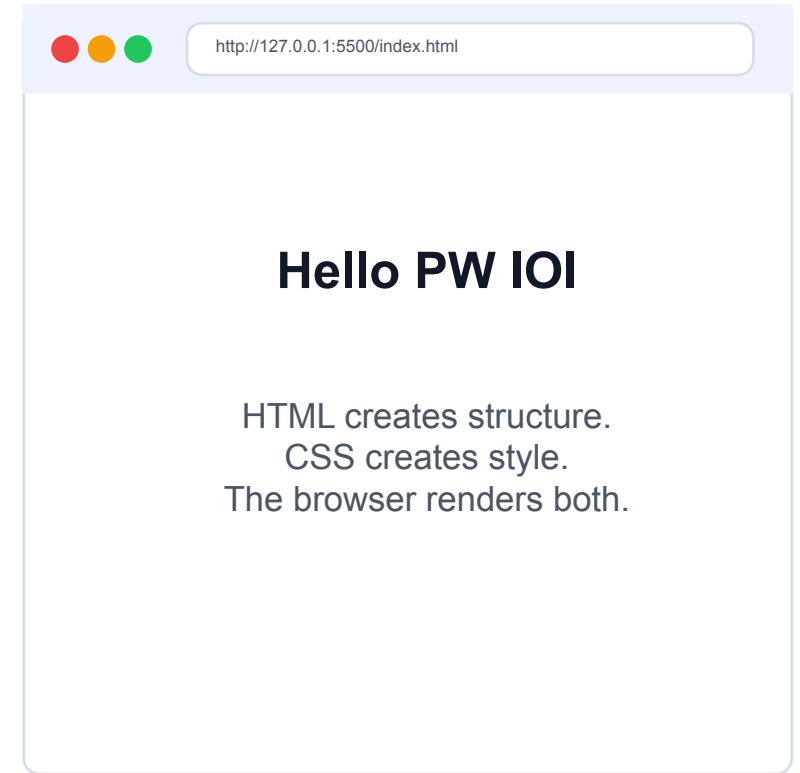


Web Development Tools & HTML Foundations

A beginner-friendly classroom deck for understanding tools, browser internals, HTML structure, elements, metadata, colors and paths.

PW IOI

@adfarsiofficial



Learning Map

We move from tools → websites → browser internals → HTML syntax → paths and colors.

1. Tools

Node.js, npm, VS Code, extensions, server

2. Core web words

Website, webpage, static, dynamic, page composition

3. Browser internals

Browser engine, rendering engine, networking, storage

4. HTML foundations

Parsing, DOM, element types, head, body, meta

5. Practical syntax

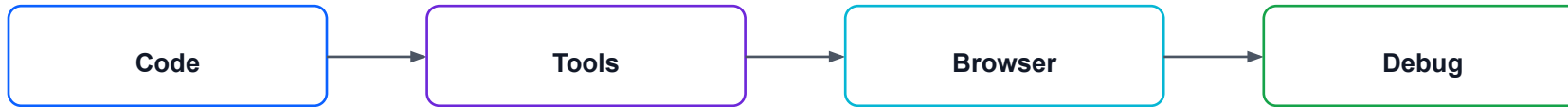
Hello World, colors, paths, project structure

Class goal

By the end, students should be able to explain what happens when an HTML file opens in the browser, read the Elements panel in DevTools, and write a clean basic HTML page.

Web Development Tools

The basic setup used to build and test websites



Editor

A code editor is where we write project files like index.html, style.css and app.js. VS Code is the most common editor for beginners.

Runtime

Node.js lets JavaScript run outside the browser. It is used for tools, servers and package scripts.

Package manager

npm, pnpm or yarn downloads reusable code packages and stores them in node_modules.

Browser + DevTools

The browser displays the website. DevTools helps us inspect HTML, CSS, network requests and errors.

Node.js

JavaScript runtime used heavily in modern web development

Definition

Node.js is a runtime environment that allows JavaScript to run on your computer or server, not only inside a browser.

Why we need it

Modern frontend tools like Vite, React, Tailwind, TypeScript and build scripts usually run using Node.js.

Where it is used

Backend APIs, command-line tools, development servers, automation scripts and package installation.

Simple check

After installing Node, open terminal and run: `node -v` and `npm -v`. If versions appear, setup is working.

```
node -v  
npm -v  
node app.js
```

Package Managers

Tools that install, update and manage project dependencies

Definition

A package manager downloads external libraries and records them in package.json so the project can be recreated later.

npm

Comes with Node.js by default. Most beginner projects use npm because it is already installed.

package.json

The file that stores project name, scripts and dependencies. It is like the project instruction file.

node_modules

The folder where installed packages are kept. We usually do not edit this folder manually.

```
npm init -y
npm install vite
npm run dev
```

Installation of Node.js

Practical setup flow for students

Step 1

Open the official Node.js website and download the LTS version. LTS means Long Term Support and is safer for most learners.

Step 2

Run the installer and keep default settings unless your instructor says otherwise. npm is installed with Node.js.

Step 3

Open terminal and verify installation using `node -v` and `npm -v`. Version numbers confirm setup.

Step 4

Create a test file `app.js` and run `node app.js` to confirm JavaScript runs outside the browser.

```
// app.js
console.log("Node is working");

// terminal
node app.js
```

VS Code

The editor where students write and organize web projects

Definition

VS Code is a code editor with file explorer, terminal, extensions, formatting and debugging support.

Important panels

Explorer for files, Editor area for code, Terminal for commands, Extensions for extra features.

Beginner workflow

Open a folder, create index.html, write code, save the file, then open it in a browser.

Best practice

Always open the project folder, not only a single file. This helps relative paths and Live Server work properly.

```
project-folder/  
  index.html  
  style.css  
  images/  
    logo.png
```

Extensions in VS Code

Extensions add useful features but should not replace fundamentals

Live Server

Starts a local development server and refreshes the browser when files change. Useful for HTML/CSS practice.

Prettier

Formats code consistently so indentation and spacing stay clean.

HTML CSS Support

Improves suggestions and linking between HTML and CSS files.

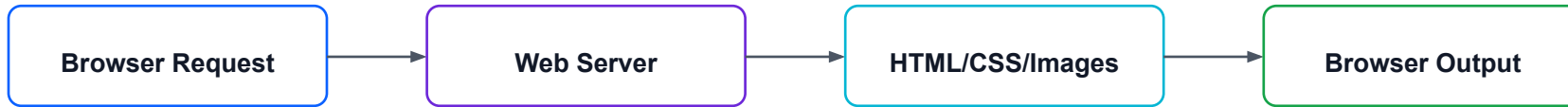
Auto Rename Tag

When you rename an opening HTML tag, the closing tag is renamed automatically.

```
Right click index.html  
→ Open with Live Server  
  
Browser opens:  
http://127.0.0.1:5500/index.html
```

Web Server

A server receives requests and sends responses



Definition

A web server is software or hardware that accepts HTTP requests and returns files or data as HTTP responses.

Example

When the browser requests `/index.html`, the server sends the HTML file. It may also send CSS, images and JavaScript.

Local server

Live Server is a small local web server used during learning. It runs on your own computer.

Production server

A deployed website uses a real server or hosting platform so users can access it over the internet.

Website

A collection of related webpages under one domain

Definition

A website is a complete collection of pages, assets and features available through a domain or URL.

Example

pw.live can have home page, course page, login page, dashboard page and payment page. Together they form a website.

Main parts

Pages, navigation, media files, styles, scripts, server logic and database content.

Simple idea

A website is like a complete book. Each webpage is like one page of that book.

Web Page

One document displayed in the browser

Definition

A webpage is a single HTML document rendered by the browser. It may include CSS, images, fonts and JavaScript.

Example

/about.html is one page. /contact.html is another page. In React apps, routes behave like pages.

What it contains

Text, headings, links, images, forms, lists, sections and other elements.

Browser role

The browser reads the HTML, builds the DOM, applies CSS and paints pixels on the screen.

Static Pages

Same content is sent to every user

Definition

A static page is a page whose HTML content is already written and does not change based on user data at request time.

Example

A portfolio about page, a simple landing page or a documentation page can be static.

Good for

Fast pages, simple websites, marketing pages and content that does not change often.

Limitation

Static pages cannot show personalized dashboard data unless extra dynamic logic or APIs are added.

Dynamic Pages

Content can change based on data, user or time

Definition

A dynamic page is generated or updated using data. Different users may see different content.

Example

A student dashboard shows enrolled courses, progress and assignments for the logged-in student.

How it works

The browser or server fetches data from an API or database and renders content accordingly.

Simple comparison

Static page: same menu for everyone. Dynamic page: your orders, your profile, your marks.

Page Composition

How a webpage is divided into meaningful sections

Definition

Page composition means arranging parts of a page so information is easy to read and use.

Common sections

Header, navigation, hero section, content area, cards, forms, sidebar and footer.

HTML thinking

Each visible section should map to meaningful HTML elements like header, main, section, article and footer.

Good composition

A good page has clear hierarchy: title first, important action next, supporting details after that.

Evolution of HTML

HTML became more semantic and web-app friendly over time

Early HTML

Focused mainly on documents, headings, paragraphs, links and images. Layout was often table-based.

HTML4/XHTML era

More structure and stricter syntax were introduced, but rich web apps still needed plugins.

HTML5

Introduced semantic tags, audio/video, canvas, forms, storage APIs and better support for modern applications.

Today

HTML is the structure layer of almost every website and web app, even when using React or Next.js.

Browser Engine

The high-level system that coordinates browser work

Definition

A browser engine is the core part of a browser that coordinates loading pages, parsing files, running engines and displaying content.

It connects

Networking, rendering engine, JavaScript engine, storage, security and the browser user interface.

Examples

Chromium-based browsers use Blink for rendering and V8 for JavaScript. Firefox uses Gecko and SpiderMonkey.

Simple analogy

The browser engine is like a project manager. It does not do every job itself, but coordinates all major jobs.

Rendering Engine

The part that converts HTML and CSS into pixels



Definition

The rendering engine reads HTML and CSS, builds internal trees, calculates layout, paints visual parts and sends the result to the screen.

Key output

It decides what appears, where it appears, how big it is and how it visually looks.

Main stages

HTML parsing → DOM, CSS parsing → CSSOM, DOM + CSSOM → render tree, layout, paint, composite.

Important point

The rendering engine does not simply display raw HTML. It transforms code into visual pixels step by step.

JS Engine

The part that runs JavaScript code

Definition

A JavaScript engine reads, compiles and executes JavaScript code. It powers interactivity and logic.

Examples

V8 is used in Chrome and Node.js. SpiderMonkey is used in Firefox. JavaScriptCore is used in Safari.

Role in webpage

It can update the DOM, respond to clicks, validate forms, fetch data and change page content after loading.

Not same as rendering

The JS engine runs logic. The rendering engine displays the page. They work together but are different parts.

Networking in Browser

How the browser fetches files from the internet or local server

Definition

Networking is the browser process of making requests and receiving responses over protocols like HTTP and HTTPS.

Example flow

Browser requests index.html. HTML mentions style.css and logo.png. Browser then sends more requests for those files.

DevTools place

The Network tab shows each request, status code, file size and loading time.

Class demo

Open a page → Inspect → Network → Refresh. Students can see every file the browser downloaded.

Data Persistence

Ways browsers and servers remember data

Definition

Data persistence means data remains available after page refresh, browser close or server restart.

Browser storage

localStorage, sessionStorage, cookies and IndexedDB store data on the user device.

Server storage

Databases like MongoDB or PostgreSQL store data for the application on the backend.

Example

A theme preference can be stored in localStorage. A user account should be stored in a database.

Programming vs Markup Language

HTML describes structure; programming languages describe logic

Markup language

A markup language uses tags to describe meaning and structure of content. HTML tells the browser what each piece of content is.

Programming language

A programming language can create logic using variables, conditions, loops, functions and algorithms.

HTML example

`<h1>Course Title</h1>` means this text is the main heading.

JavaScript example

if logged in, show dashboard. This is logic, so it belongs to a programming language.

```
<!-- HTML marks structure -->
<h1>Hello</h1>

// JavaScript writes logic
if (isLoggedIn) showDashboard();
```

HTML Parsing in the Browser

Parsing means the browser reads HTML text and converts it into a DOM tree.

```
<!DOCTYPE html>
<html>
  <head><title>Demo</title></head>
  <body>
    <h1>Hello</h1>
    <p>Welcome to class</p>
  </body>
</html>
```

Step 1

1. Browser receives HTML text

Step 2

2. Tokenizer reads tags and text

Step 3

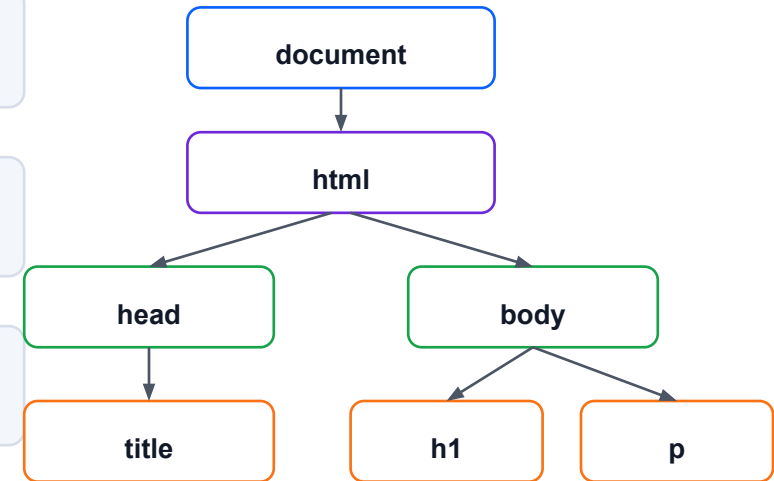
3. Parser creates nodes

Step 4

4. Nodes connect as parent and child

Step 5

5. Final output is the DOM tree

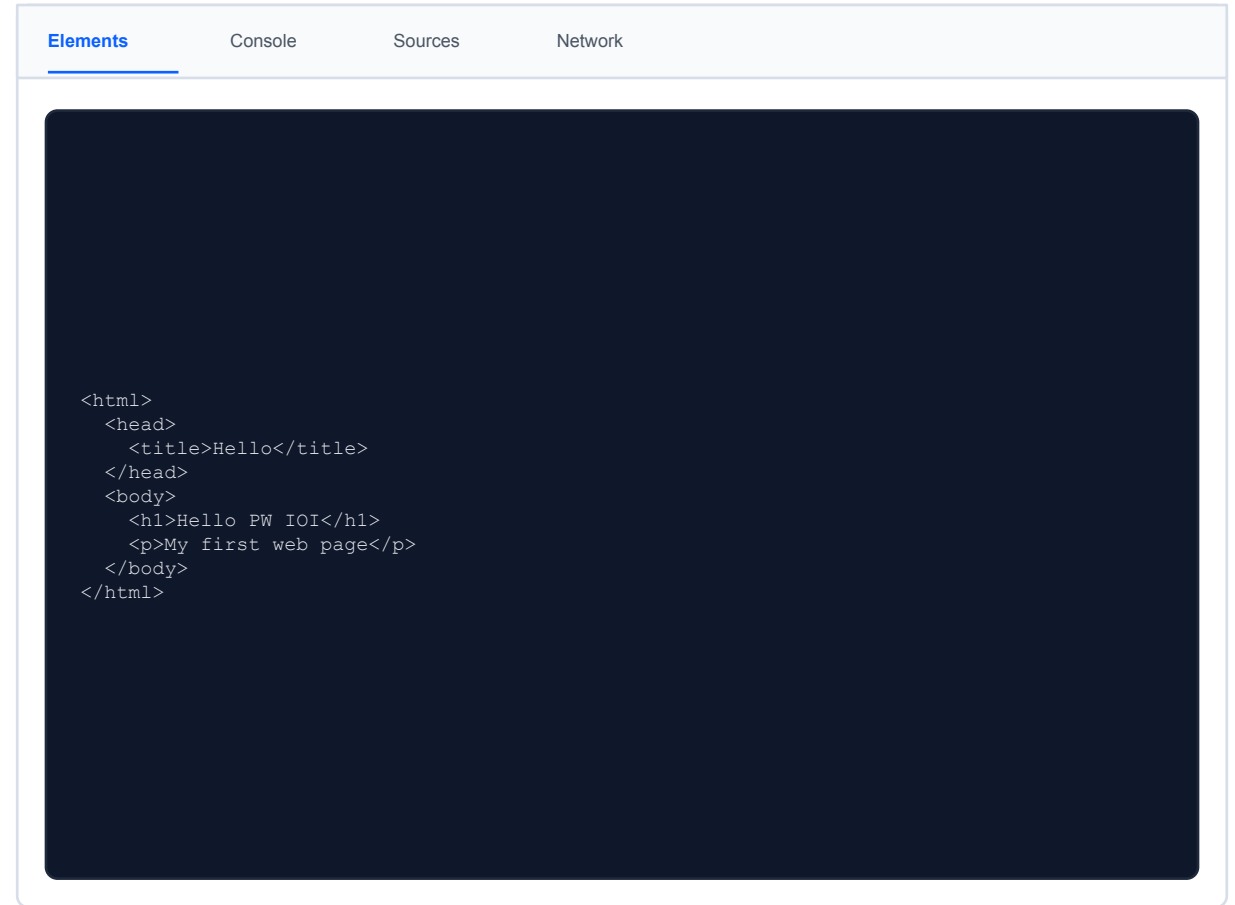
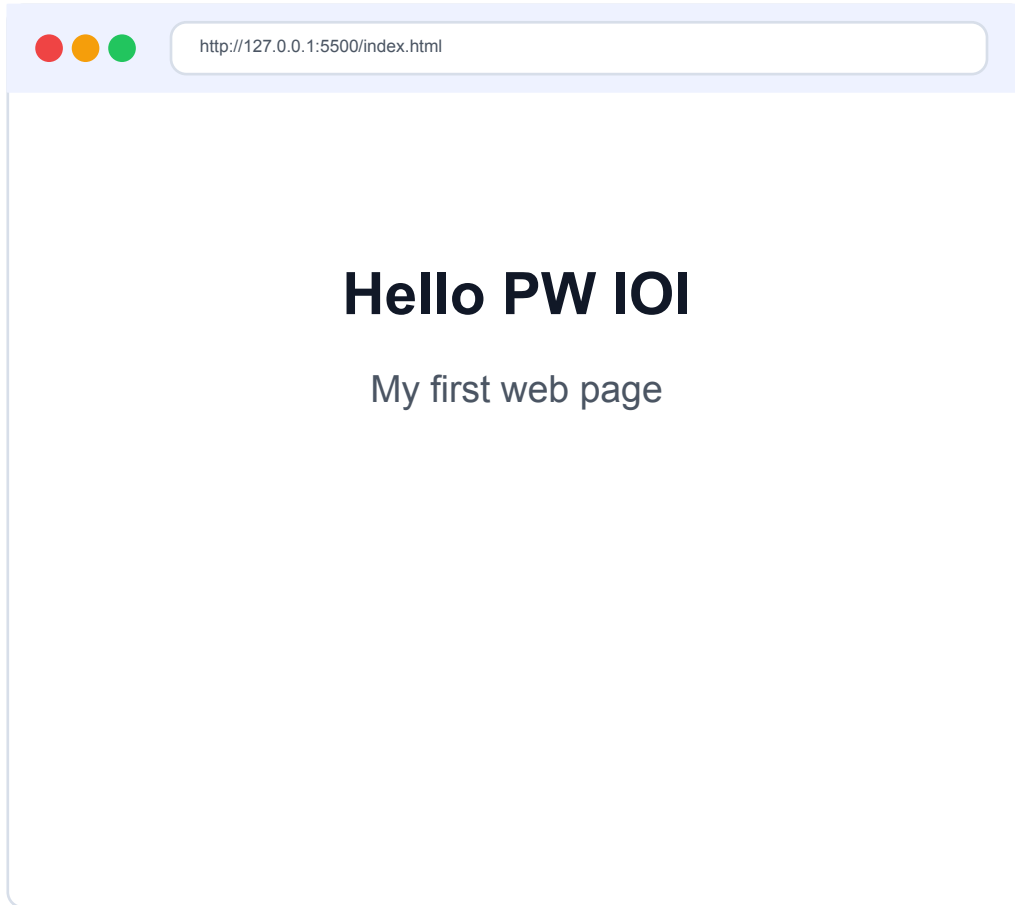


Easy explanation

When you open inspect and look at the Elements panel, you are not looking at plain source text only. You are seeing the DOM-like structure that the browser created after parsing HTML.

Inspect → Elements Panel

A practical way to show HTML parsing output to students.

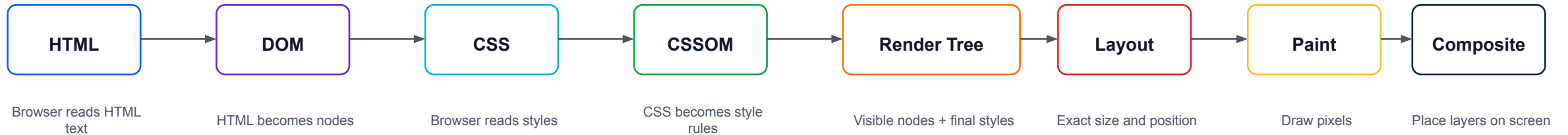


How to show in browser

1. Open index.html with Live Server.
2. Right click on page → Inspect.
3. Open the Elements tab.
4. Expand html, head and body.
5. Explain that this tree is created after HTML parsing.

Rendering Flow: From Code to Screen

The complete browser rendering pipeline in beginner-friendly steps.



```
HTML: <h1 class="title">Hello</h1>
CSS:  .title { color: blue; font-size: 40px;
}
```

Simple output

The rendering engine combines the h1 element and its CSS rule, calculates where it should appear, paints blue text, and finally shows it on the screen.

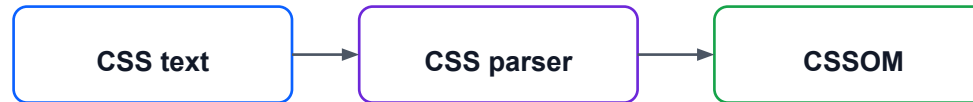
Key line for class

HTML parsing creates structure. CSS parsing creates styles. Rendering uses both to create the final visual output.

CSSOM: CSS Object Model

CSSOM is the browser's internal tree of CSS rules.

```
body { margin: 0; font-family: Arial; }  
h1 { color: blue; font-size: 40px; }  
p { color: gray; }
```



What CSSOM stores

It stores which CSS rules apply, how selectors match elements and which final style values should be used.

Why it matters

Without CSSOM, the browser knows the content but does not know how it should look: colors, spacing, fonts and sizes.

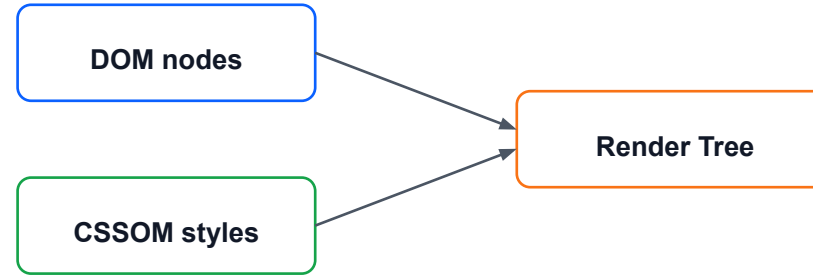
Example

h1 element from DOM + h1 rule from CSSOM = heading with blue color and 40px font size.

Render Tree

The render tree contains only visible things that need to be painted.

```
<body>
  <h1>Hello</h1>
  <p>Visible paragraph</p>
  <p style="display:none">Hidden</p>
</body>
```



What goes inside

Visible elements like body, h1 and visible p become render tree nodes with calculated styles attached.

What may not go inside

Elements with display:none are in the DOM, but they are not part of the render tree because they are not painted.

Class explanation

DOM answers: what exists? CSSOM answers: how should it look? Render tree answers: what visible boxes should be drawn?

Layout, Paint and Composite

After the render tree, the browser turns boxes into pixels.

Layout / Reflow

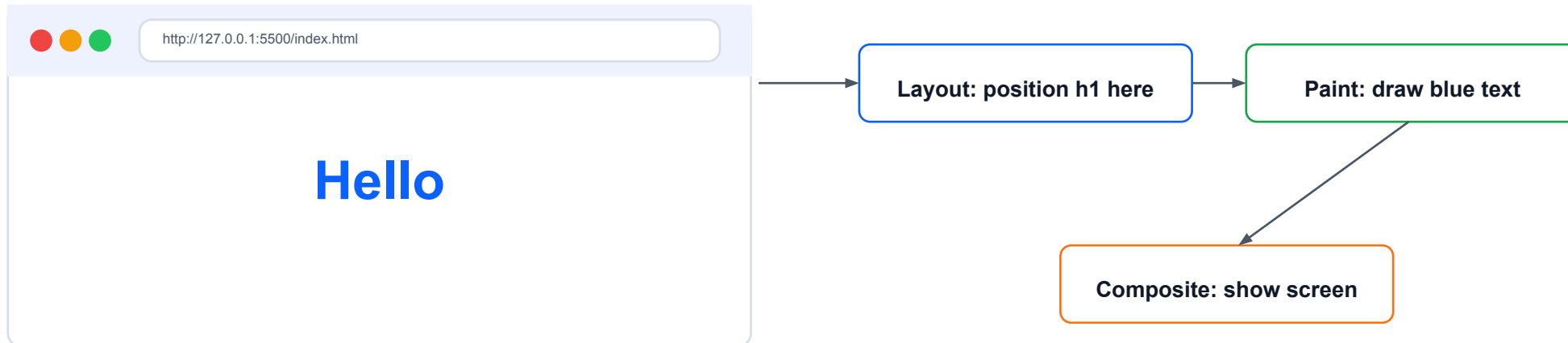
The browser calculates the exact width, height, x-position and y-position of each visible box. Example: h1 starts 20px from top and takes 300px width.

Paint

The browser draws visual parts: text color, background color, borders, shadows and images. Example: draw blue h1 text on white background.

Composite

The browser combines painted layers and sends final result to the screen. This helps with scrolling, animations and fixed elements.



DOM Overview

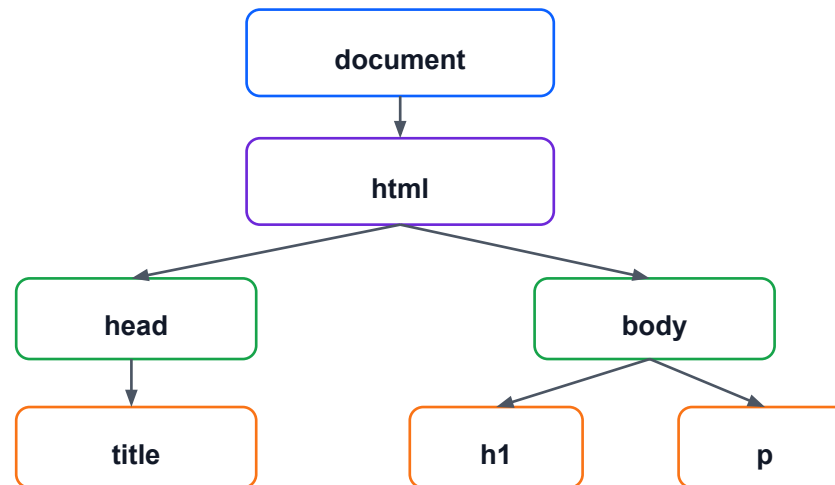
DOM means Document Object Model. It is the browser-created tree of the page.

Definition

DOM is a tree-like representation of the HTML document created by the browser. Each tag becomes a node that can be read or changed.

Why useful

DevTools Elements panel shows this tree. JavaScript can later use DOM APIs to add, remove or update elements.



Easy line

HTML Element Types

HTML has different element categories because the browser treats their content differently.

Normal elements

Have opening and closing tags and can contain text or other elements. Example: `<p>Hello</p>`, `<div>...</div>`.

Void elements

Do not wrap content and do not need closing tags. Example: `<img>`, `<br>`, `<hr>`, `<input>`.

Text-special elements

Raw text and RCDATA elements are parsed differently because their inside content is mostly treated as text.

Foreign elements

Elements from another language inside HTML, mainly SVG and MathML. Example: `<svg>...</svg>`.

Normal Elements

Most HTML elements are normal elements.

Definition

A normal element has a start tag, content and an end tag. The browser creates a node and also parses the content inside it.

Syntax

`<tagname>content</tagname>`. The closing tag tells the browser where that element ends.

Examples

`<h1>Title</h1>`, `<p>Paragraph</p>`, `<div>Box</div>`, `<section>Content</section>`.

Important

If closing tags are missing, the browser may auto-correct the DOM, but students should write clean complete HTML.

```
<section>
  <h2>Frontend</h2>
  <p>HTML gives structure to the page.</p>
</section>
```

Void Elements

Elements that cannot contain child content.

Definition

A void element represents something that exists by itself. It has no inner content and no separate closing tag.

Examples

`<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`.

Correct syntax

Use ``. Do not write `<img>photo</img>`.

Why special

The browser knows these elements are complete as soon as it reads the start tag.

```

<br>
<hr>
<input type="text" placeholder="Enter name">
```

Raw Text Elements

Elements where inner content is treated as raw text.

Definition

Raw text elements tell the HTML parser not to parse normal HTML tags inside their content. The content is treated as text until the closing tag.

Important examples

`<style>` and `<textarea>` are commonly discussed in parsing behavior. For this class, focus on style because JavaScript is not part of this deck right now.

Style example

Inside `<style>`, CSS code is not parsed as HTML tags. It is handed to the CSS parser.

Student warning

Do not confuse raw text with visible text. Raw text means parser behavior, not whether users can see it.

RCDATA Elements

RCDATA means Replaceable Character Data.

Definition

In RCDATA elements, normal HTML tags are mostly treated as text, but character references like `&` can still be decoded.

Common examples

`<title>` and `<textarea>` are the most common examples to explain.

Example

`<title>PW & IOI</title>` becomes the title text “PW & IOI”.

Why it matters

It explains why the title tag contains text for the browser tab instead of creating visual page elements.

```
<title>PW &amp; IOI</title>
<!-- Browser tab text: PW & IOI -->
```

Foreign Elements

SVG and MathML inside HTML.

Definition

Foreign elements are elements from another markup language embedded inside HTML. The browser switches parsing rules for that region.

SVG example

`<svg><circle cx="50" cy="50" r="40"></circle></svg>` creates a vector circle.

MathML example

MathML can represent mathematical notation in browsers that support it.

Why special

They are not normal HTML elements, but the browser can still place them inside the DOM.

```
<svg width="100" height="100">
  <circle cx="50" cy="50" r="40"></circle>
</svg>
```

Writing Hello World in HTML

A complete beginner HTML file with visible browser output

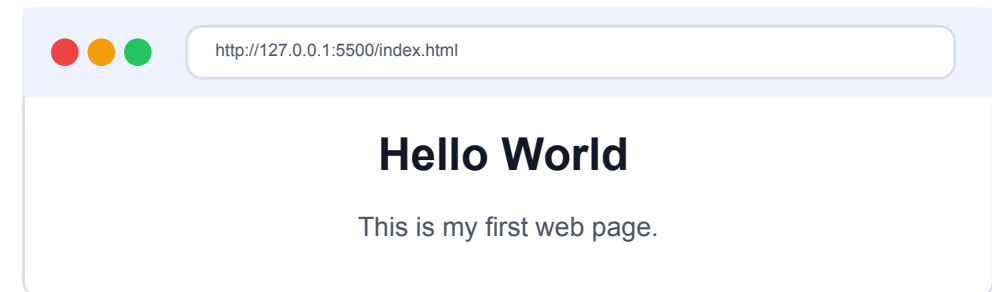
```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello World</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

How to run

Create index.html → paste this code → save → open with Live Server or double click the file.

Browser output

The h1 and p tags appear inside the page. The title appears in the browser tab.



head Section

Information about the page, mostly not visible inside the page body

```
<head>
  <title>My Page</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
</head>
```

Definition

The head section stores metadata, title, external links and configuration that helps the browser understand the document.

Visible or invisible?

Most head content is not displayed inside the page. It affects tab title, SEO, encoding, responsiveness and linked files.

body Section

The part that contains visible page content

```
<body>
  <header>Course Website</header>
  <main>
    <h1>Learn Full Stack</h1>
    <p>Start with HTML foundations.</p>
  </main>
  <footer>© 2026</footer>
</body>
```

Definition

The body contains the content users see and interact with: headings, paragraphs, images, buttons, lists and forms.

Best practice

Keep content meaningful and structured. Use semantic elements when possible.

head Section: title and link

title controls browser tab text; link attaches external resources

```
<title>PW IOI HTML Class</title>  
<link rel="stylesheet" href="style.css">  
<link rel="icon" href="favicon.ico">
```

title

The title appears in the browser tab and is also used by search engines and bookmarks.

link

The link element connects external files like CSS stylesheets, favicons or preloaded assets. It is a void element.

Meta Tags

Small instructions that describe the page to browsers and search engines

```
<meta charset="UTF-8">
<meta name="description" content="HTML basics class">
<meta name="keywords" content="HTML, CSS, Web
Development">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<meta http-equiv="refresh" content="10">
```

charset

Tells the browser how to decode characters. UTF-8 supports English, Hindi, Urdu, emojis and most characters.

viewport

Makes responsive pages work properly on mobile screens. It tells the browser to match page width with device width.

style Tag

Used to write internal CSS inside the HTML file

```
<style>
  body { font-family: Arial; }
  h1 { color: blue; }
</style>
```

Definition

The style tag contains CSS rules for the current HTML document. The browser sends this content to the CSS parser.

Best practice

For small demos, style tag is okay. For larger projects, keep CSS in a separate style.css file and link it.

Deprecated body attributes

bgcolor, text and background are old HTML styling attributes

```
<!-- Old style: avoid in modern HTML -->
<body bgcolor="yellow" text="blue" background="bg.jpg">

<!-- Modern way: use CSS -->
<body>
<style>
  body { background-color: yellow; color: blue; }
</style>
```

bgcolor

Old attribute used to set page background color. Today we use CSS background-color.

text

Old attribute used to set default text color. Today we use CSS color.

background

Old attribute used to set background image. Today we use CSS background-image.

Why avoid

HTML should describe structure. CSS should handle design. This keeps code clean and maintainable.

HTML Colors

Colors can be written using names, HEX, RGB, HSL and CSS variables

```
<h1 style="color: red">Red text</h1>
<h1 style="color: #0B61FF">HEX blue</h1>
<h1 style="color: rgb(16, 42, 67)">RGB navy</h1>
<h1 style="color: hsl(221, 100%, 52%)">HSL blue</h1>
```

Named colors

Easy for learning: red, blue, green, black, white. Limited but readable.

HEX

Common in design systems: #0B61FF.
Two digits each for red, green and blue.

RGB

Uses numeric red, green and blue values from 0 to 255.

HSL

Uses hue, saturation and lightness.
Useful when creating color variations.

Relative and Absolute Paths

Paths tell the browser where to find files

```
<!-- Relative path: from current file location -->

<link rel="stylesheet" href="css/style.css">

<!-- Absolute URL: full address -->

```

Relative path

A relative path starts from the current file or current website folder. Best for local project assets.

Absolute path / URL

An absolute URL gives the full address including protocol and domain. Useful for external resources.

Common mistake

If the image does not load, check folder name, file spelling, extension and whether you opened the correct project folder.

Quick demo

Open DevTools → Network. A missing file usually shows 404, meaning the browser requested it but could not find it.

Folder Structure

A clean project folder makes paths easier to understand and debug.

```
my-first-website/  
  index.html  
  about.html  
  css/  
    style.css  
  images/  
    logo.png  
    banner.jpg
```

Why structure matters

The browser follows paths exactly. If index.html says images/logo.png, then an images folder with logo.png must exist next to index.html.

Example linking CSS

If style.css is inside css folder, link it as css/style.css from index.html.

Example image path

If logo.png is inside images folder, use images/logo.png from index.html.

Debug tip

Use DevTools Network tab. If a file is red or shows 404, the path is wrong or the file is missing.

Class Recap

The most important ideas students should remember.

Tools

VS Code to write code, Node/npm for tooling, Live Server for local preview.

Web basics

Website is a collection. Webpage is one document. Static is fixed. Dynamic changes with data.

Browser internals

HTML becomes DOM. CSS becomes CSSOM. DOM + CSSOM become render tree, then layout, paint and composite.

HTML basics

head stores metadata. body stores visible content. Elements can be normal, void, RCDATA, raw text or foreign.

Practical debugging

Use Inspect → Elements to see DOM and Network tab to debug missing files.

Practice task:

Create `index.html` with title, meta viewport, linked CSS, one heading, one paragraph, one image and correct relative paths.